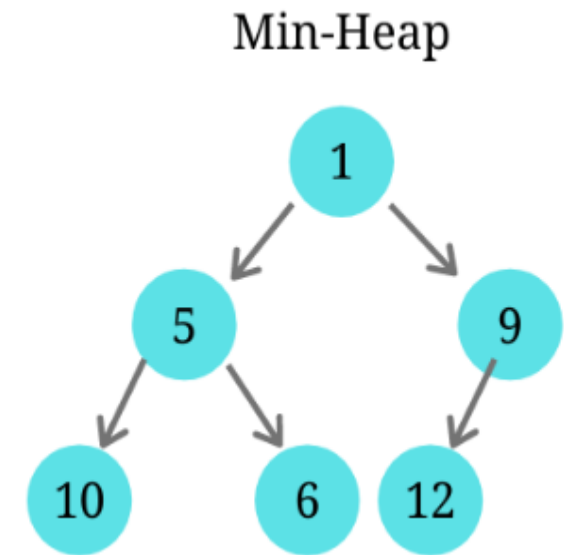
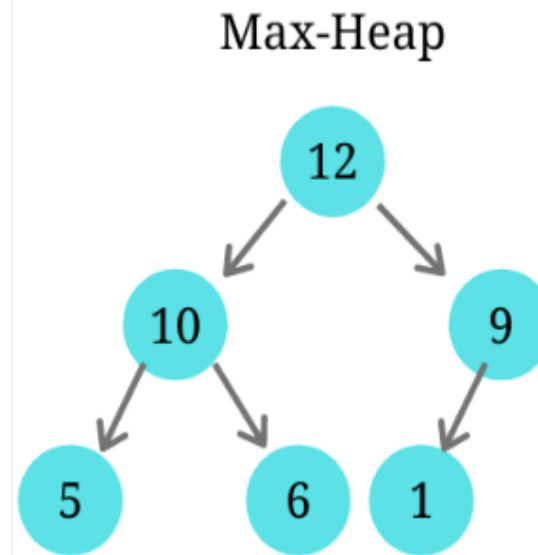


Module-7 : Heap



Dr. Durgesh Kumar
Assistant Professor (Senior),
SCOPE, VIT Vellore

Lecture Outline

❑ Heap Introduction

- ❑ Definition & Properties – max Heap, min Heap

- ❑ Usage/Need

- ❑ Function of MaxHeap/ MinHeap ADT

- $\text{max_heapify}(A, i)$, $\text{min_heapify}(A, i)$
- $\text{build_Max_heap}(A)$, $\text{build_Min_heap}(A)$
- $\text{Heapsort}(A)$
- $\text{delete_Max_Heap}(A)$, $\text{delete_Min_Heap}(A)$
- $\text{insert}(A, x)$

❑ Practice problems of Heap

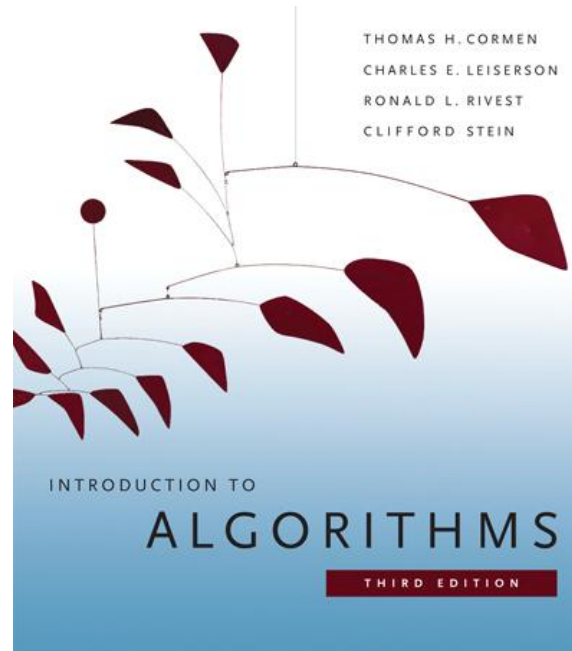
- ❑ Max Heap on integer array and character array

- ❑ Min Heap on integer array and character array

GOALS

- ❑ Understand of Maxheap, MinHeap & properties
- ❑ MaxHeap ADT functions
- ❑ Understanding Heapsort on character and integer array.
- ❑ Insertion and deletion of an element from an character/integer array.

6.006- *Introduction to Algorithms*



Courtesy of MIT Press. Used with permission.

Lecture 4

Menu

- Priority Queues
- Heaps
- Heapsort

Priority Queue

A data structure implementing a set S of elements, each associated with a key, supporting the following operations:

$\text{insert}(S, x)$: insert element x into set S

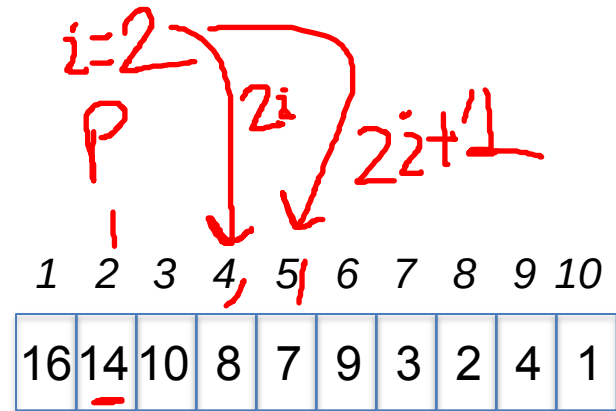
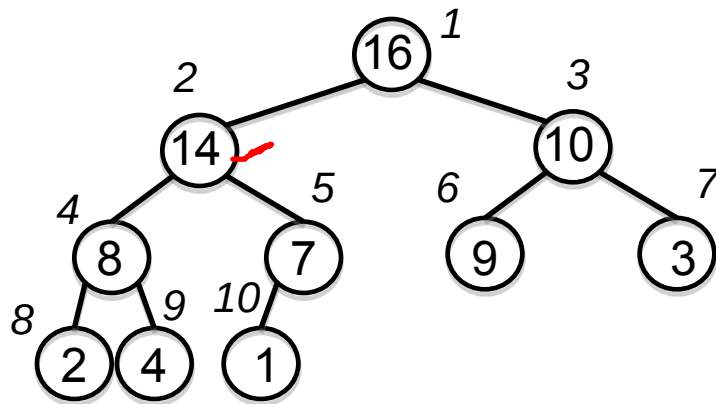
$\text{max}(S)$: return element of S with largest key

$\text{extract_max}(S)$: return element of S with largest key and remove it from S

$\text{increase_key}(S, x, k)$: increase the value of element x 's key to new value k
(assumed to be as large as current value)

Heap

- Implementation of a priority queue
- An **array**, visualized as a nearly complete **binary tree**
- **Max Heap Property**: The key of a node is $\geq$ than the keys of its children
(**Min Heap** defined analogously)



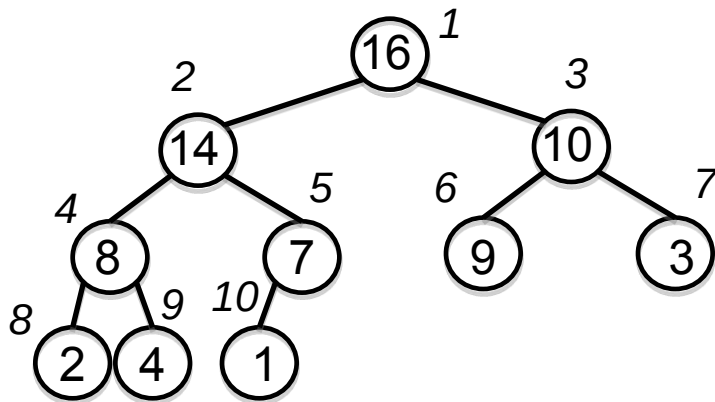
Heap as a Tree

root of tree: first element in the array, corresponding to $i = 1$

$\text{parent}(i) = i/2$: returns index of node's parent

$\text{left}(i) = 2i$: returns index of node's left child

$\text{right}(i) = 2i + 1$: returns index of node's right child



1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

No pointers required! Height of a binary heap is $O(\lg n)$

Heap Operations

build_max_heap : produce a max-heap from an unordered array

~~***~~ max_heapify : $\rightarrow \rightarrow \rightarrow \text{max_heapify}(A, i)$
correct a single violation of the heap property in a subtree at its root index i

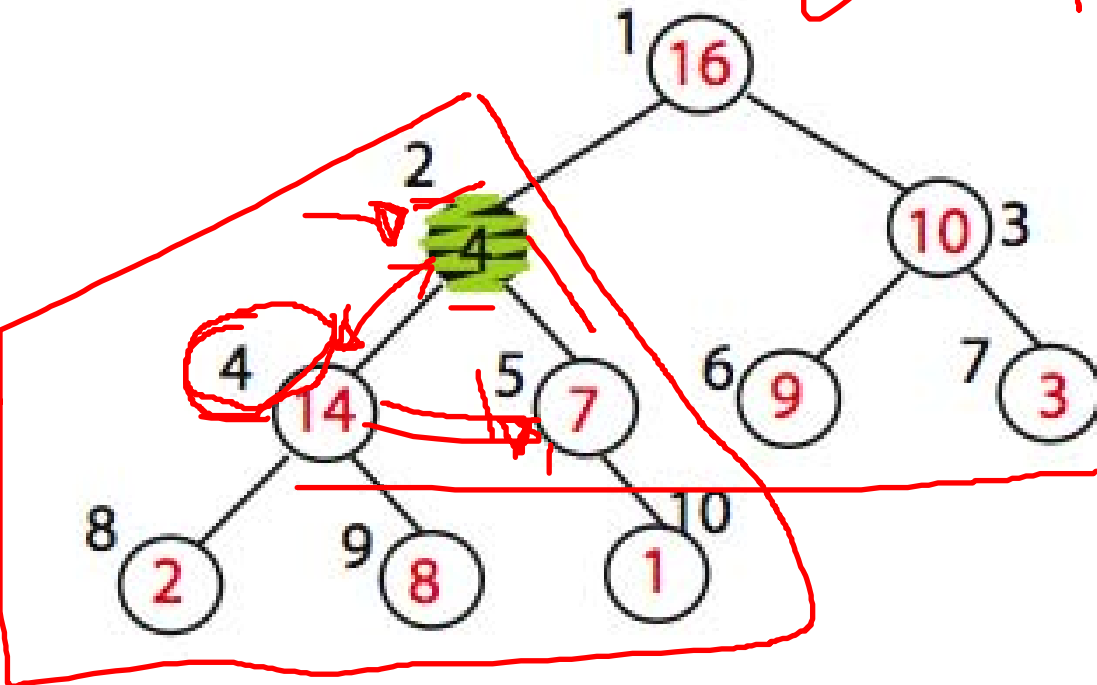
insert, extract_max, heapsort

Max_heapify

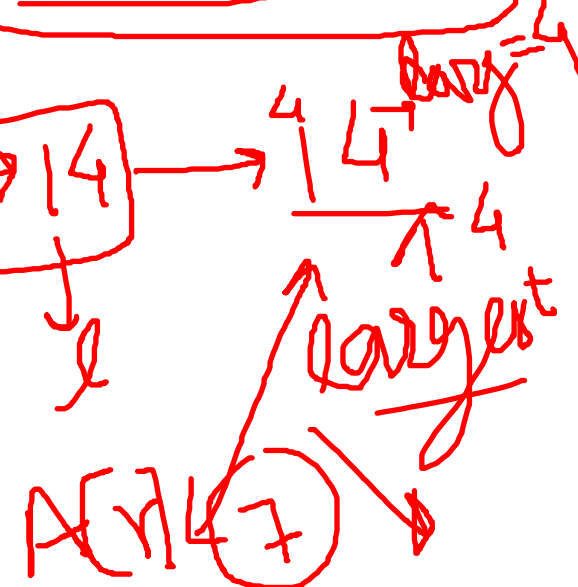
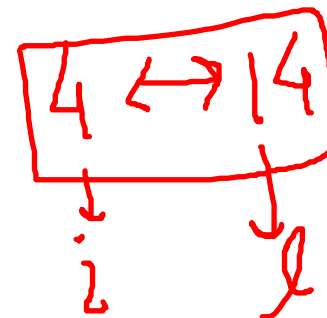
- Assume that the trees rooted at $\text{left}(i)$ and $\text{right}(i)$ are max-heaps
- If element $A[i]$ violates the max-heap property, correct violation by “trickling” element $A[i]$ down the tree, making the subtree rooted at index i a max-heap

Max_heapify (Example)

Will restore MAX-HEAP property at node
with index $i=2$

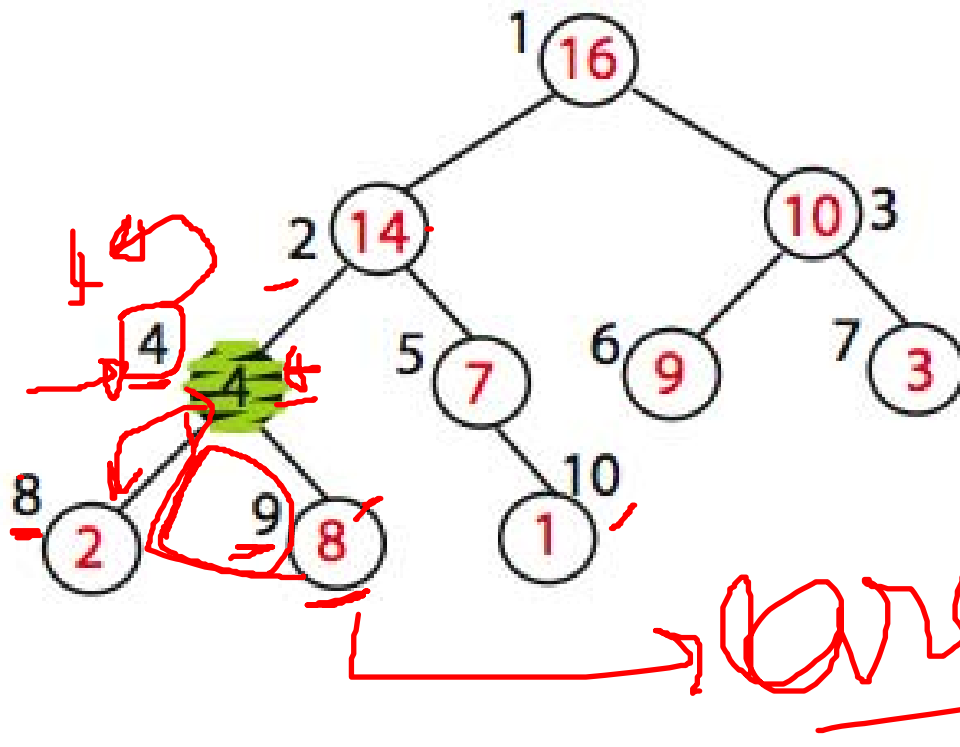


★ `MAX_HEAPIFY(A, 2)`
`heap_size[A] = 10`



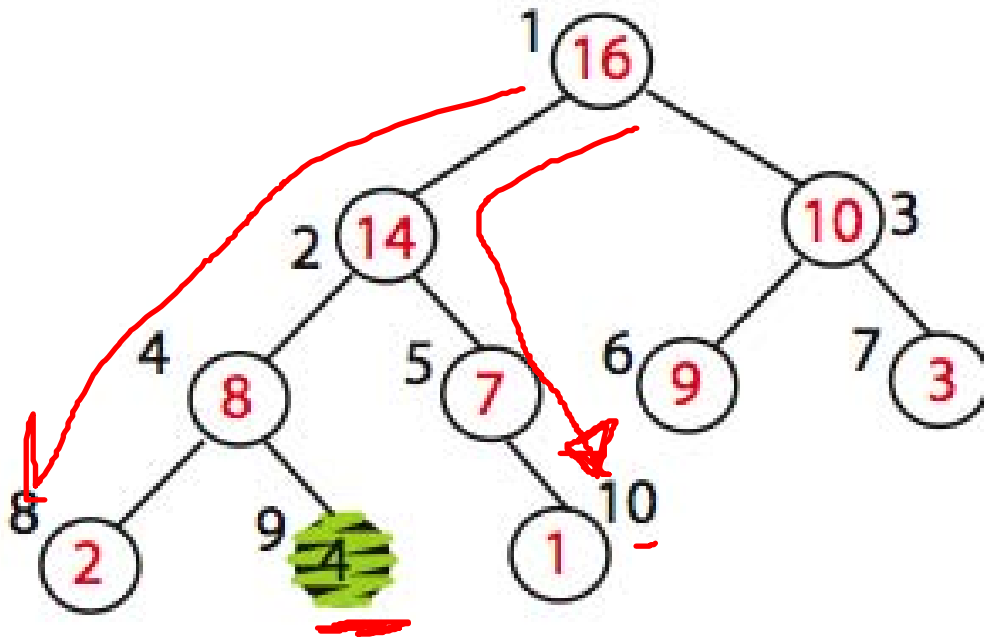
Node 10 is the left child of node 5 but is drawn to the right for convenience

Max_heapify (Example)



Exchange A[2] with A[4]
Call MAX_HEAPIFY(A,4)
because max_heap property
is violated

Max_heapify (Example)



Exchange A[4] with A[9]
No more calls

$O(\log n)$

Time=? $O(\log n)$

Max_Heapify Pseudocode

$l = \text{left}(i)$

$r = \text{right}(i)$

if ($l \leq \text{heap-size}(A)$ and $A[l] > A[i]$)

then $\text{largest} = l$ else $\text{largest} = i$

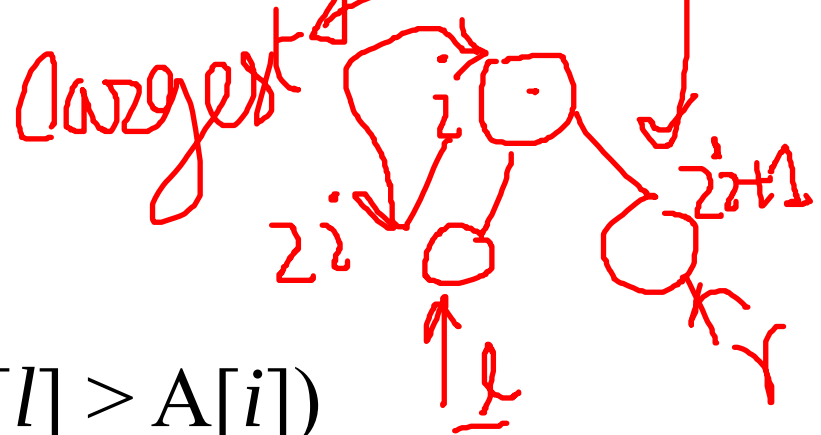
if ($r \leq \text{heap-size}(A)$ and $A[r] > A[\text{largest}]$)

then $\text{largest} = r$

if $\text{largest} \neq i$

then exchange $A[i]$ and $A[\text{largest}]$

Max_Heapify(A, largest)

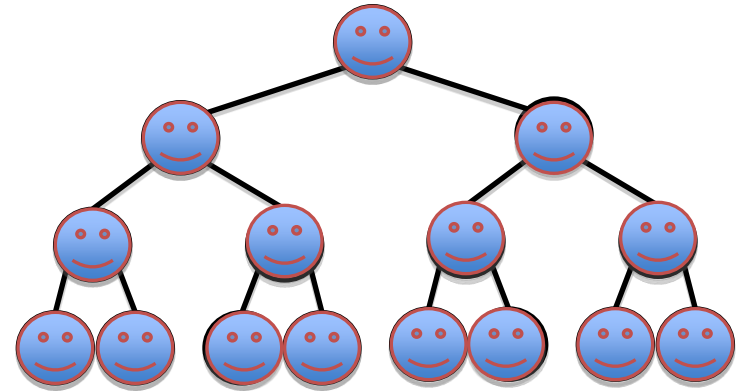


Build_Max_Heap(A)

Converts $A[1 \dots n]$ to a max heap

Build_Max_Heap(A):

for $i = n/2$ downto 1
do Max_Heapify(A, i)



$$\log(n) = \frac{n}{2} \times \log n$$
$$= \ln \log n$$

Why start at $n/2$?

Because elements $A[n/2 + 1 \dots n]$ are all leaves of the tree

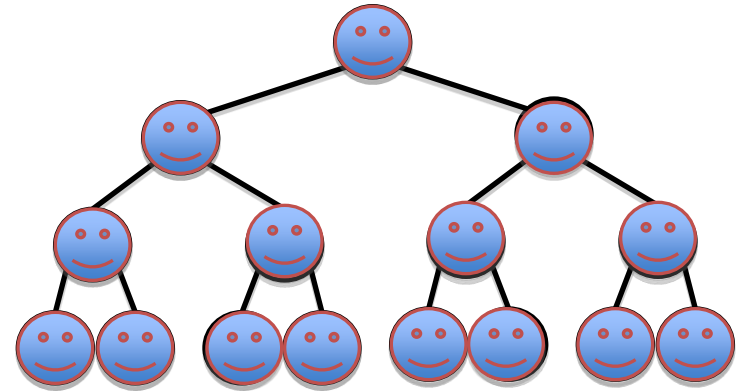
$2i > n$, for $i > n/2 + 1$

Time=? $O(n \log n)$ via simple analysis

Build_Max_Heap(A) Analysis

Converts $A[1 \dots n]$ to a max heap

```
Build_Max_Heap(A):  
  for  $i = n/2$  downto 1  
    do Max_Heapify(A, i)
```

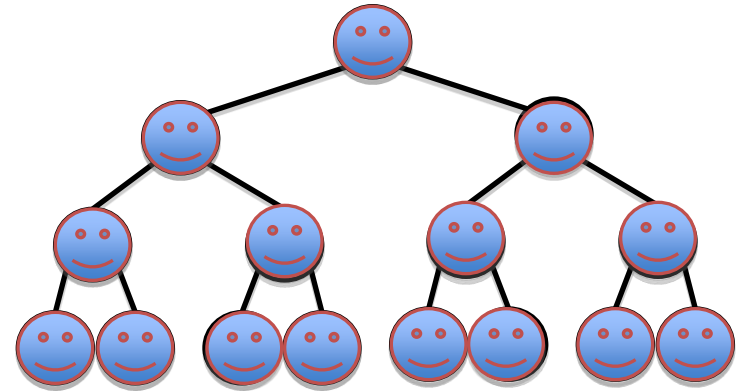


Observe however that Max_Heapify takes $O(1)$ for time for nodes that are one level above the leaves, and in general, $O(l)$ for the nodes that are l levels above the leaves. We have $n/4$ nodes with level 1, $n/8$ with level 2, and so on till we have one root node that is $\lg n$ levels above the leaves.

Build_Max_Heap(A) Analysis

Converts $A[1 \dots n]$ to a max heap

```
Build_Max_Heap(A):  
  for i=n/2 downto 1  
    do Max_Heapify(A, i)
```



Total amount of work in the for loop can be summed as:

$$n/4 (1 c) + n/8 (2 c) + n/16 (3 c) + \dots + 1 (\lg n c)$$

Setting $n/4 = 2^k$ and simplifying we get:

$$c 2^k (1/2^0 + 2/2^1 + 3/2^2 + \dots (k+1)/2^k)$$

The term in brackets is bounded by a constant!

This means that Build_Max_Heap is $O(n)$

Build-Max-Heap Demo

$i \rightarrow n/2$
 $\downarrow$
 $n/2$

A

4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---

MAX-HEAPIFY (A,5) $\rightarrow 1$

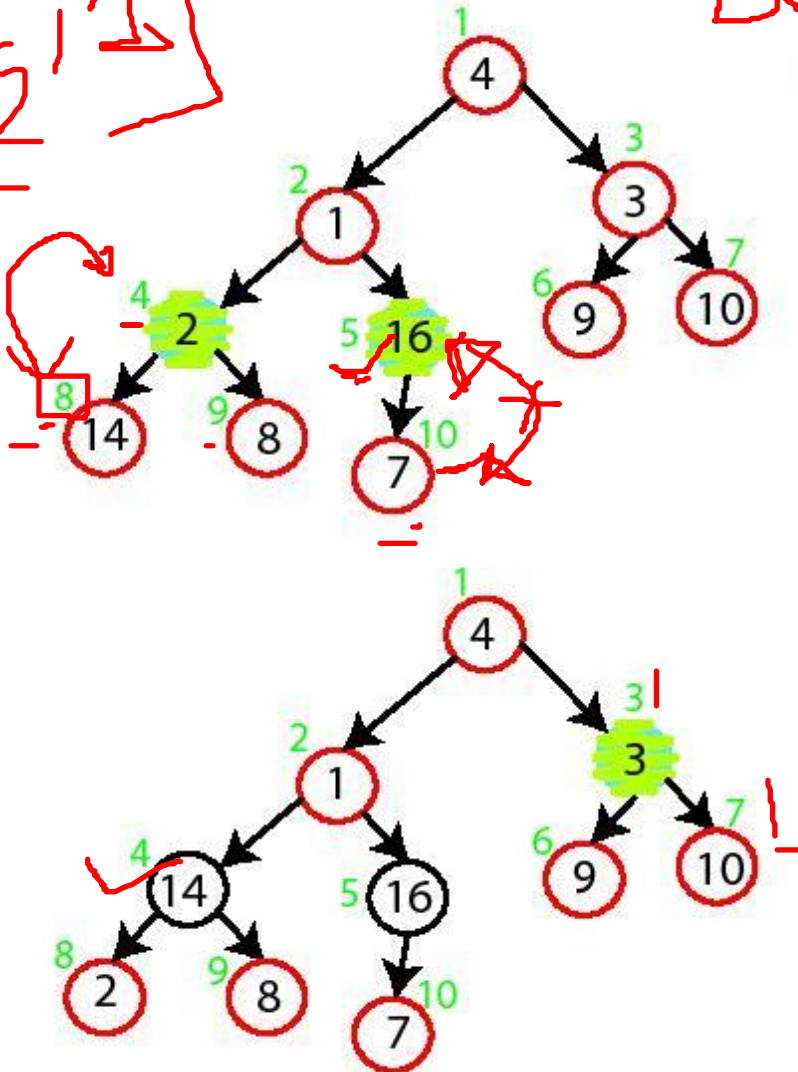
no change

MAX-HEAPIFY (A,4) ✓

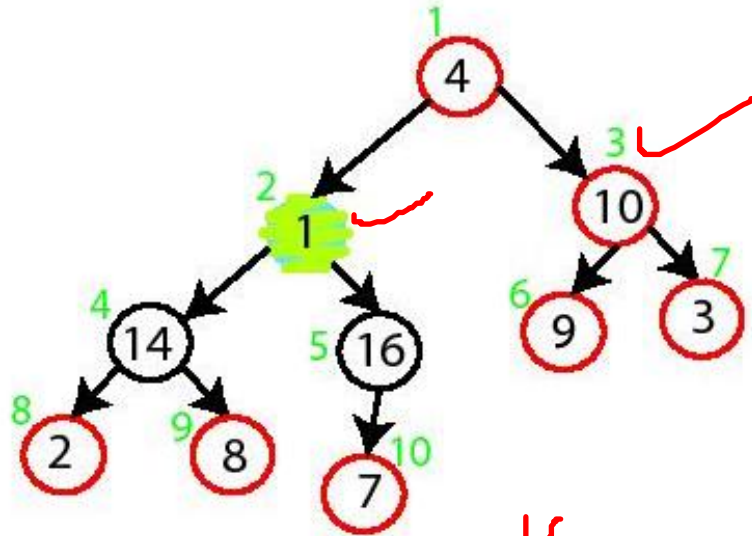
Swap A[4] and A[8]

MAX-HEAPIFY (A,3)

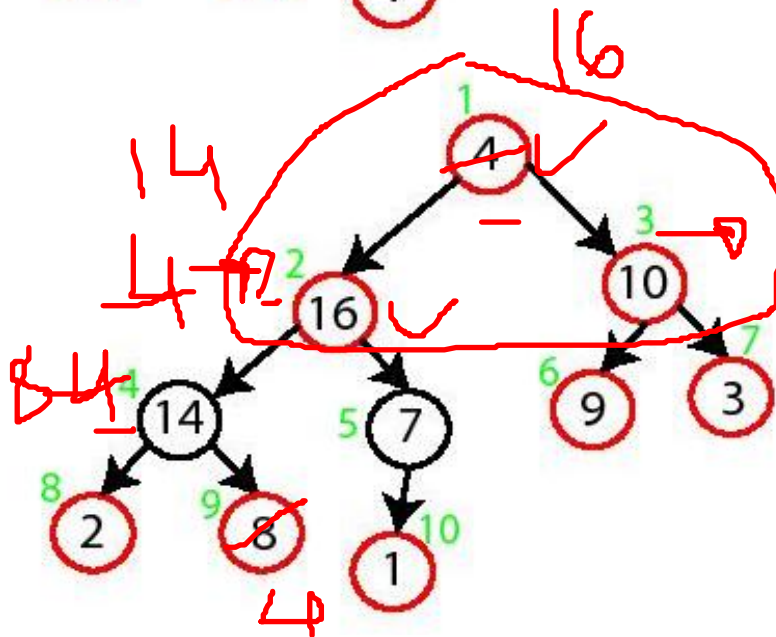
Swap A[3] and A[7]



Build-Max-Heap Demo

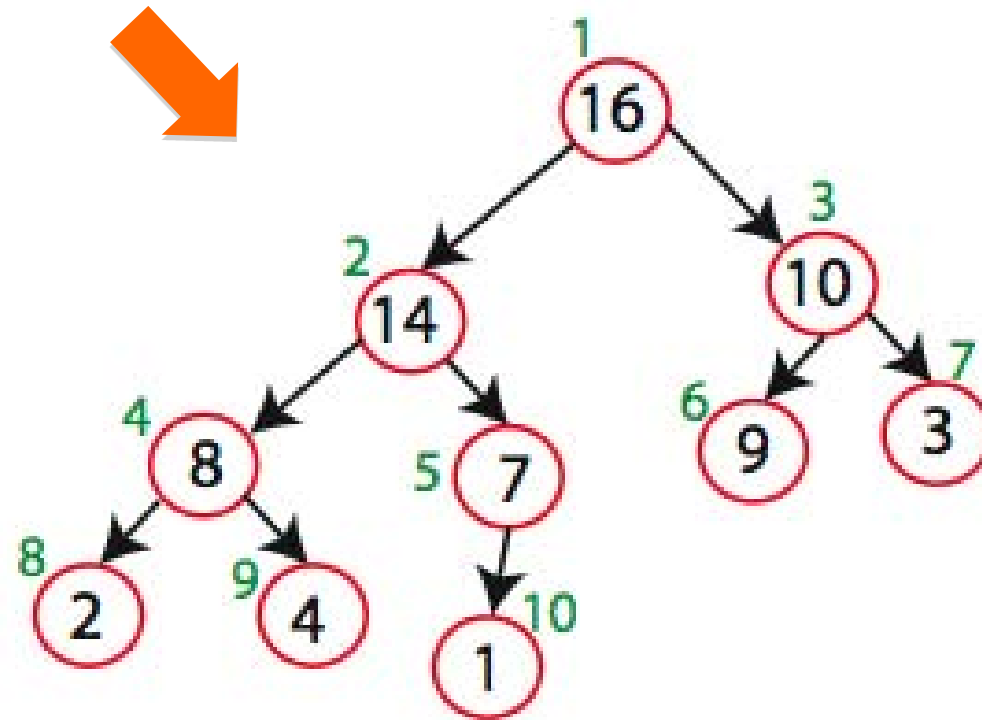


— MAX-HEAPIFY (A,2)
 Swap A[2] and A[5]
 Swap A[5] and A[10]



MAX-HEAPIFY (A,1)
 Swap A[1] with A[2]
 Swap A[2] with A[4]
 Swap A[4] with A[9]

Build-Max-Heap



Heap-Sort



Sorting Strategy:

1. Build Max Heap from unordered array;
- 

Heap-Sort

Sorting Strategy:

1. Build Max Heap from unordered array;
2. Find maximum element $A[1]$;
3. Swap elements $A[n]$ and $A[1]$:
now max element is at the end of the array!

Heap-Sort

Sorting Strategy:

1. Build Max Heap from unordered array;
2. Find maximum element $A[1]$;
3. Swap elements $A[n]$ and $A[1]$:
now max element is at the end of the array!
4. Discard node n from heap
(by decrementing heap-size variable)

Heap-Sort

Sorting Strategy:

1. Build Max Heap from unordered array;
2. Find maximum element $A[1]$;
3. Swap elements $A[n]$ and $A[1]$:
now max element is at the end of the array!
4. Discard node n from heap —
(by decrementing heap-size variable)
5. New root may violate max heap property, but its children are max heaps. Run max_heapify to fix this.

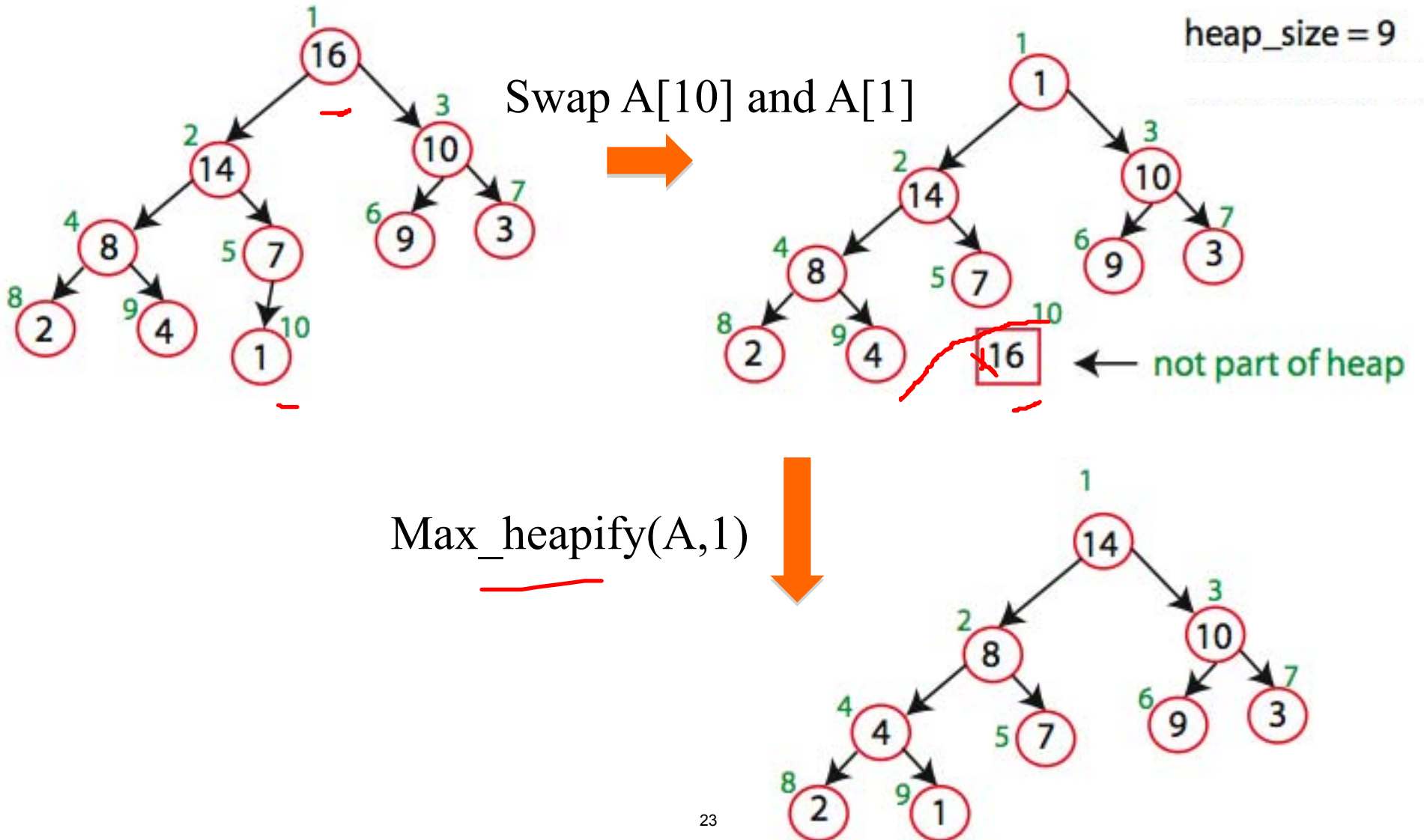


Heap-Sort

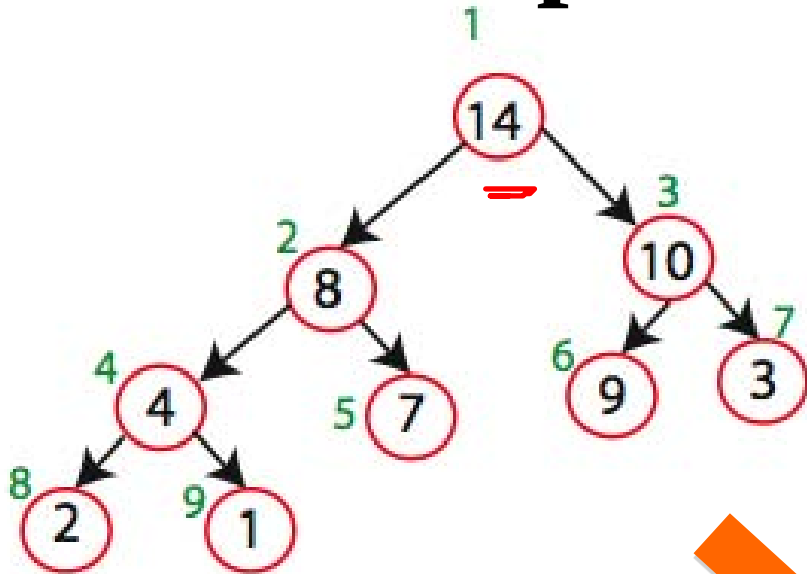
Sorting Strategy:

1. Build Max Heap from unordered array;
2. Find maximum element $A[1]$;
3. Swap elements $A[n]$ and $A[1]$:
now max element is at the end of the array!
4. Discard node n from heap
(by decrementing heap-size variable)
5. New root may violate max heap property, but its children are max heaps. Run `max_heapify` to fix this.
6. Go to Step 2 unless heap is empty.

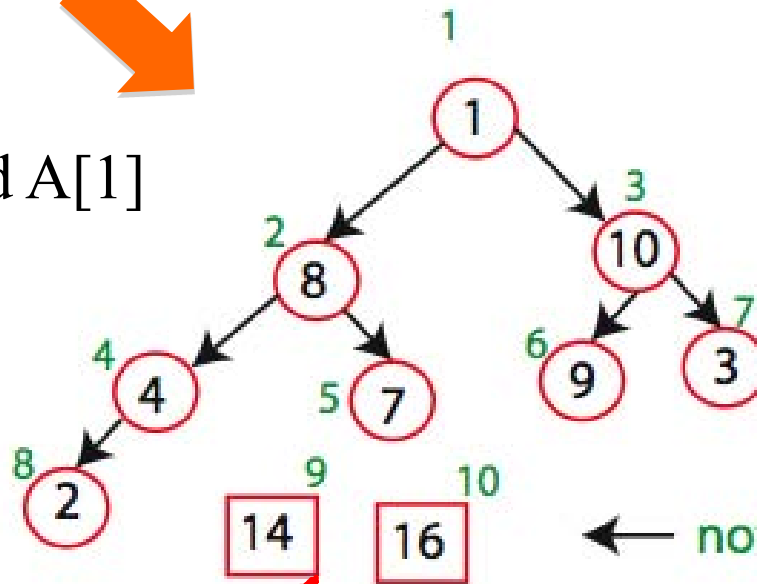
Heap-Sort Demo



Heap-Sort Demo

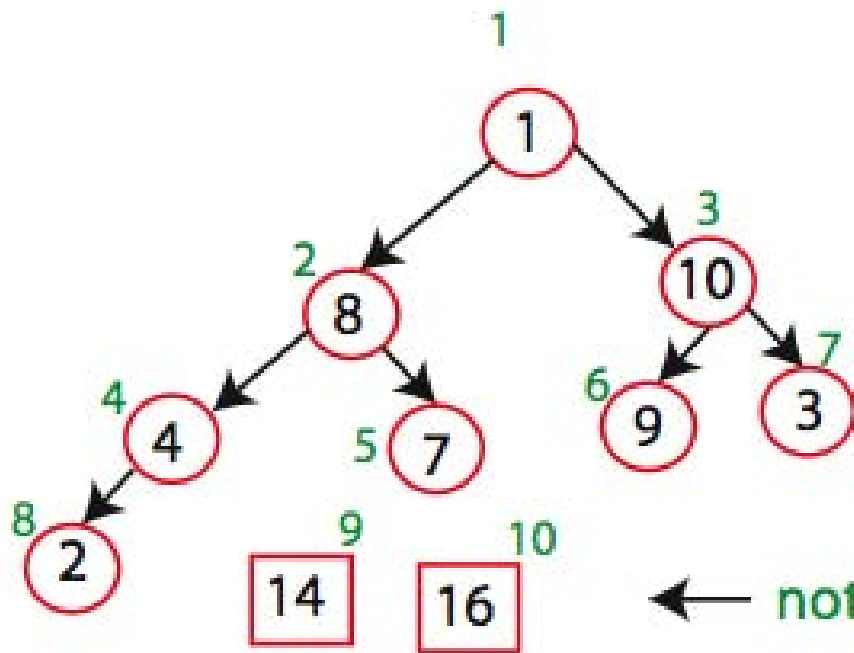


Swap $A[9]$ and $A[1]$



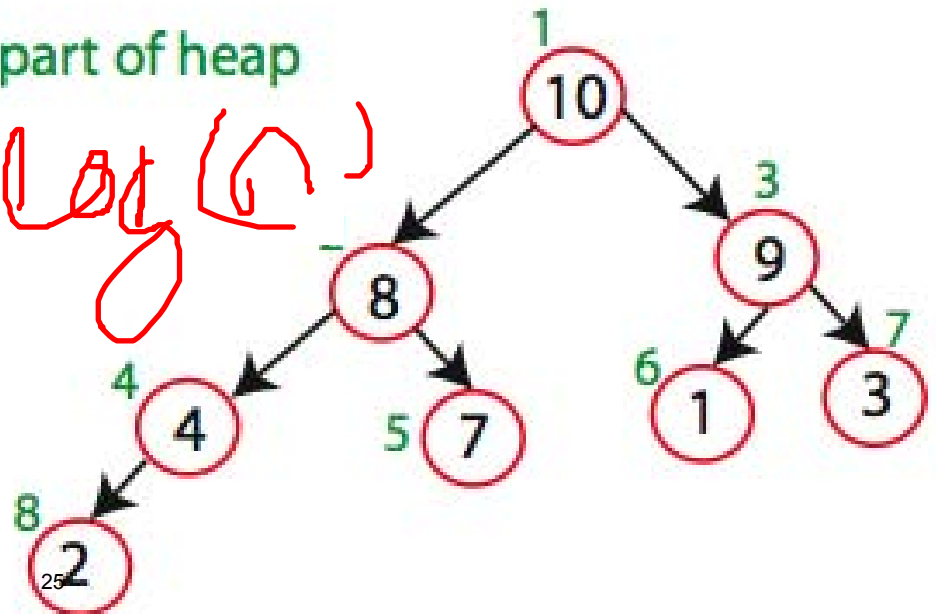
²⁴
`MAX_HEAPIFY (A,1)`

Heap-Sort Demo

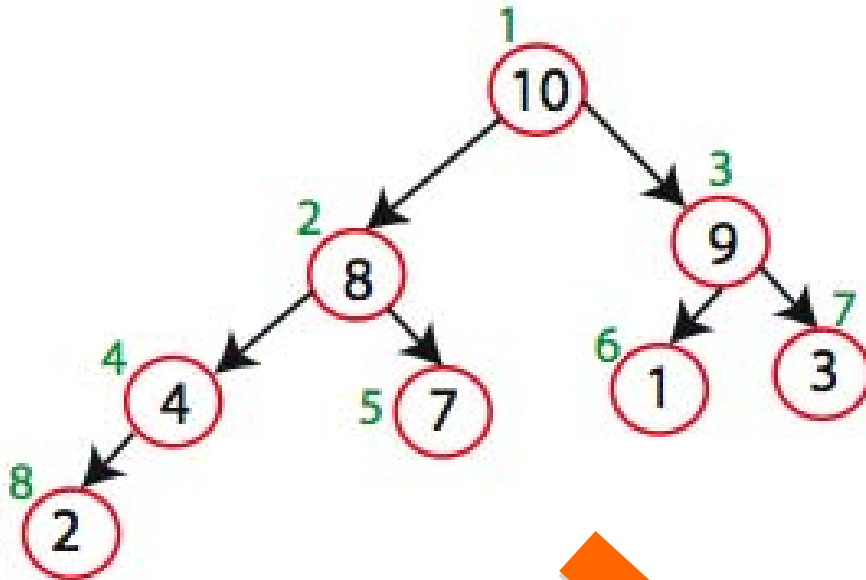


MAX_HEAPIFY (A,1)

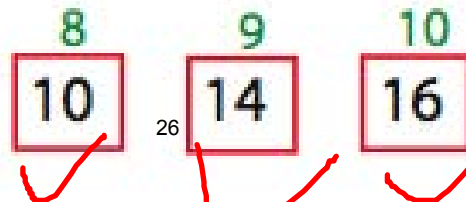
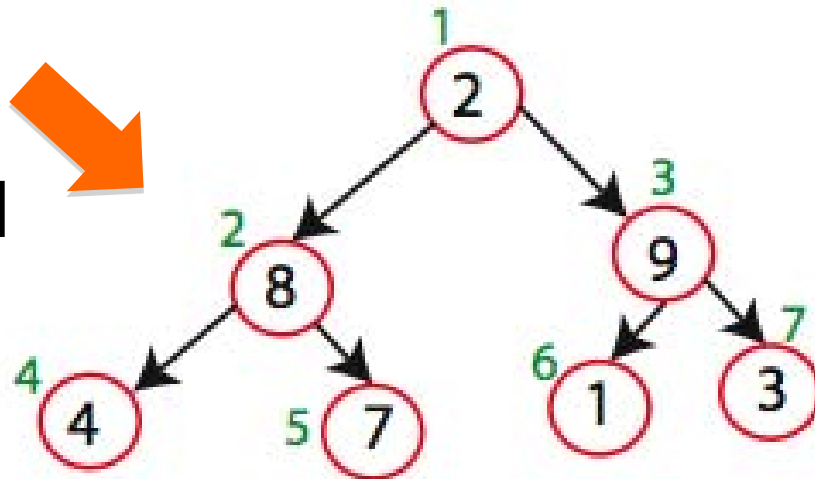
log(6)



Heap-Sort Demo



Swap $A[8]$ and $A[1]$



← not part of heap

Heap-Sort

Running time:

after n iterations the Heap is empty

every iteration involves a swap and a max_heapify operation; hence it takes $O(\log n)$ time

Overall $O(n \log n)$

MIT OpenCourseWare
<http://ocw.mit.edu>

6.006 Introduction to Algorithms
Fall 2011

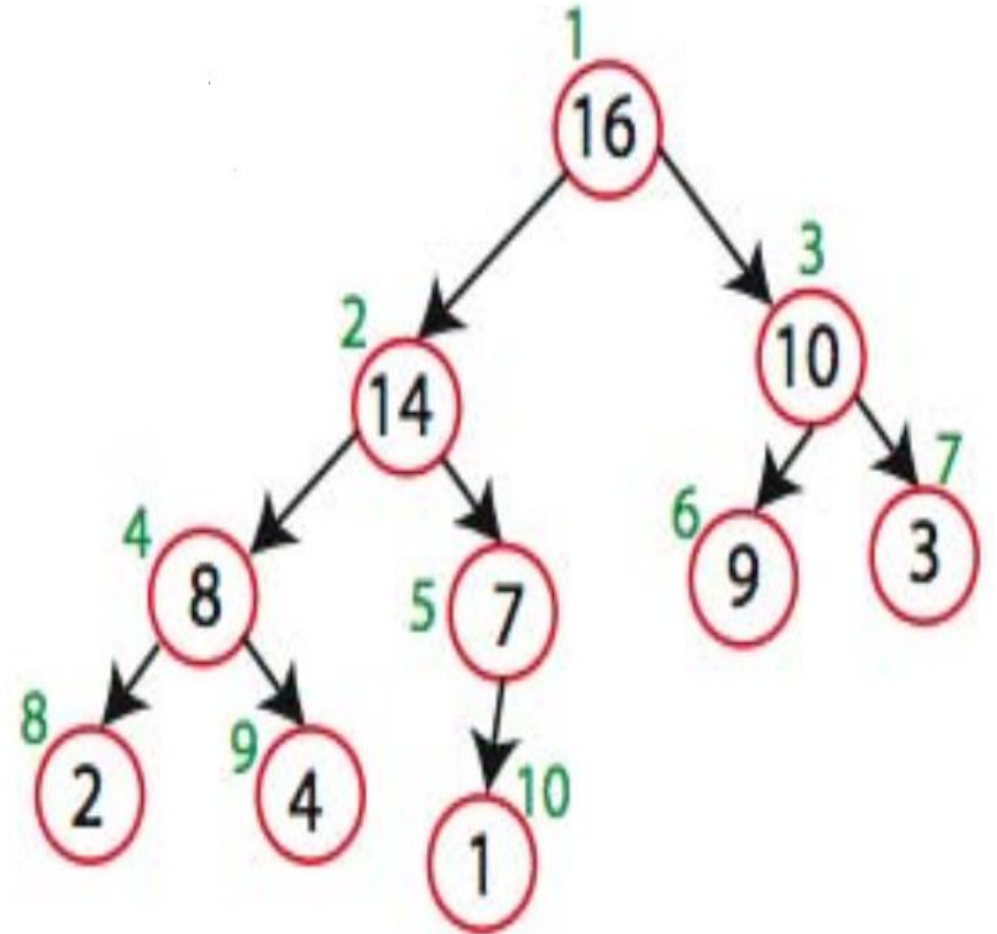
For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Heap – deletion of a new element

DeleteFromMaxHeap(A): // For MaxHeap

// We can only delete first element from Heap

1. $X = A[1]$
2. SWAP ($A[1]$, $A[n]$) // First and last element
3. Delete last element and decrease heapsize by 1.
4. Call Max_Heapify($A, 1$)
5. return x

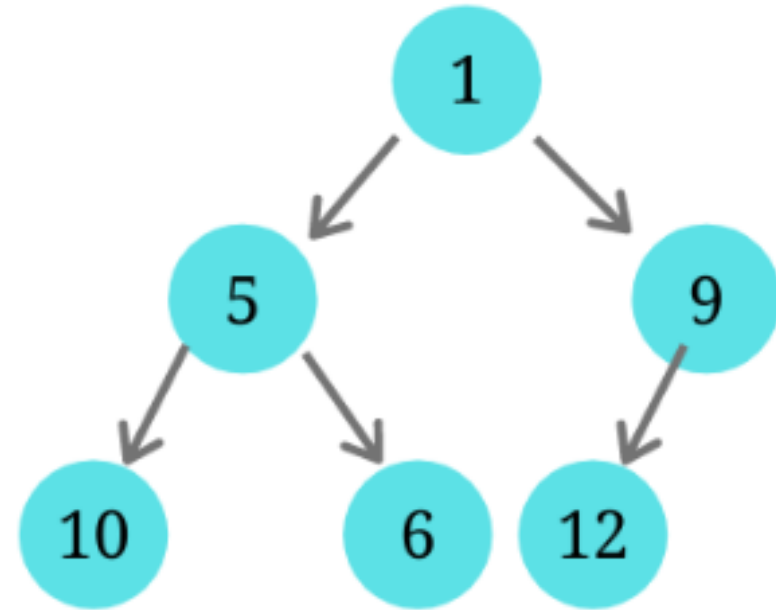


Heap – deletion of a new element

DeleteFromMinHeap(A): // For Min Heap

// We can only delete first element from Heap

1. $X = A[1]$
2. SWAP ($A[1]$, $A[n]$) // First and last element
3. Delete last element and decrease heapsize by 1.
4. Call Min_Heapify($A, 1$)
5. return x

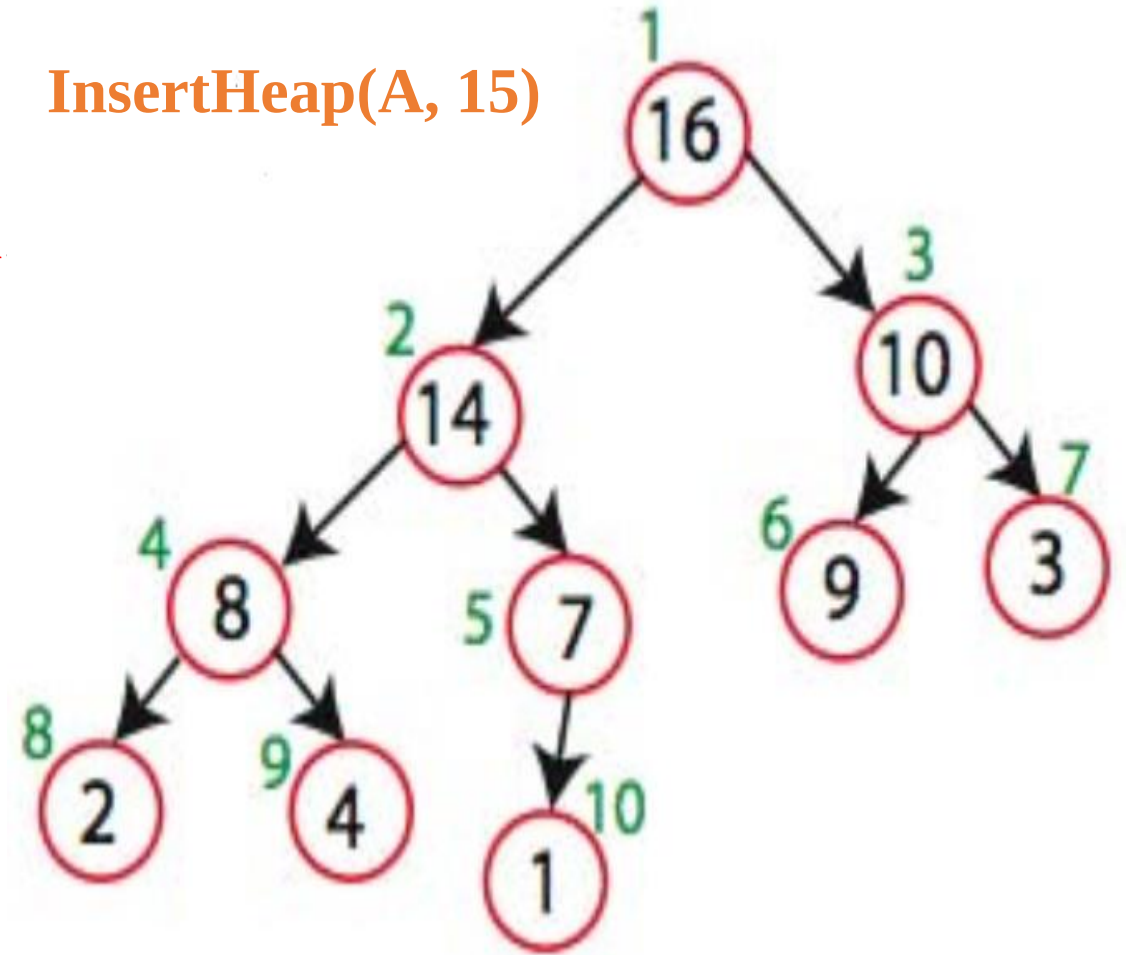


Heap –insertion of a new element

InsertHeap(A, x): // For MaxHeap

1. Increment the size of heap (n) by 1
2. Insert element **x** into array A at **last position**
3. **for**(i=n; i>1; i=i/2)
4. parent = i/2
5. // compare A[i] and A[parent]
6. **if**(A[i] < A[parent])
7. break
8. swap(A[i], A[parent])

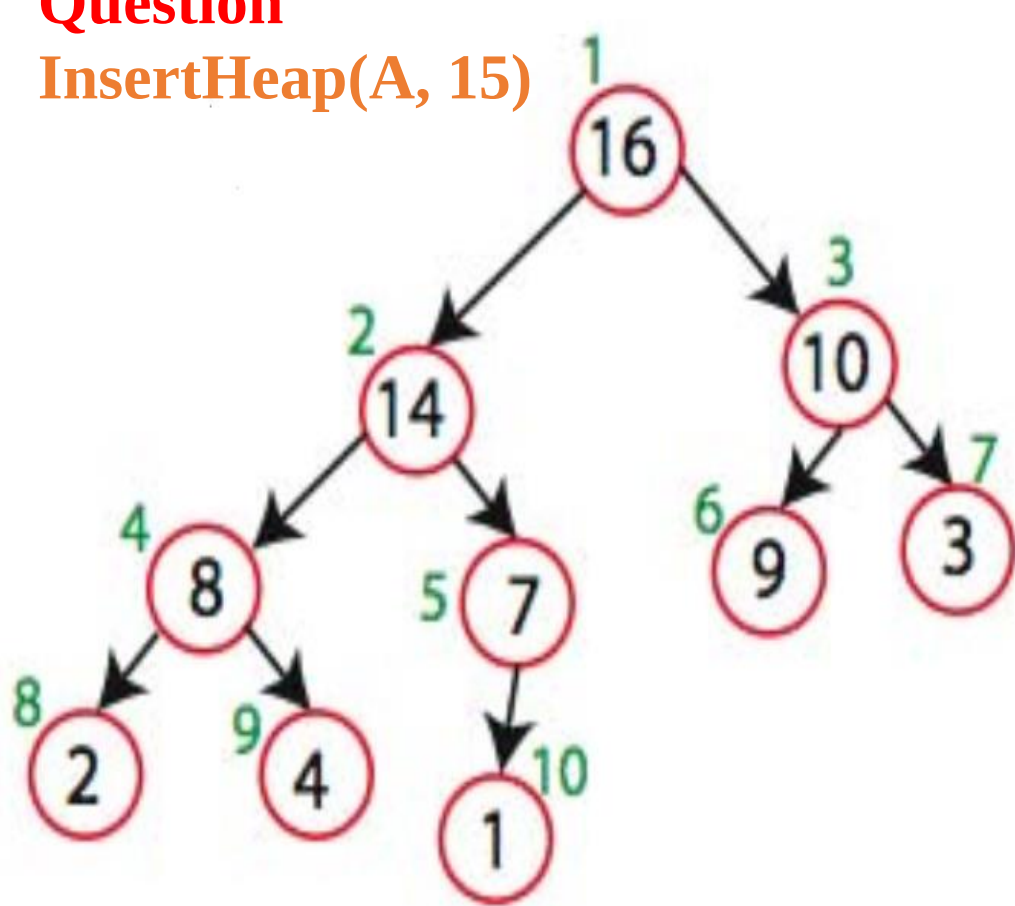
InsertHeap(A, 15)



Heap –insertion of a new element

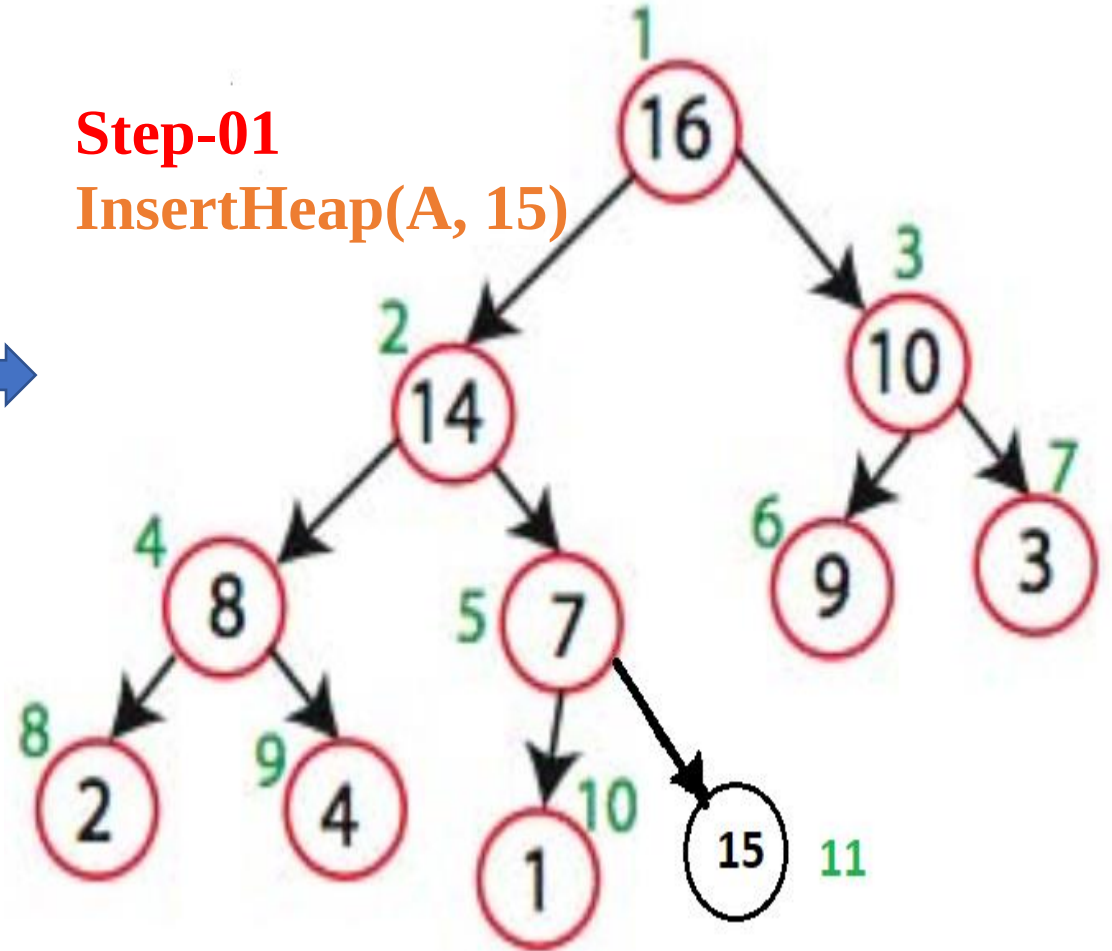
Question

InsertHeap(A, 15)



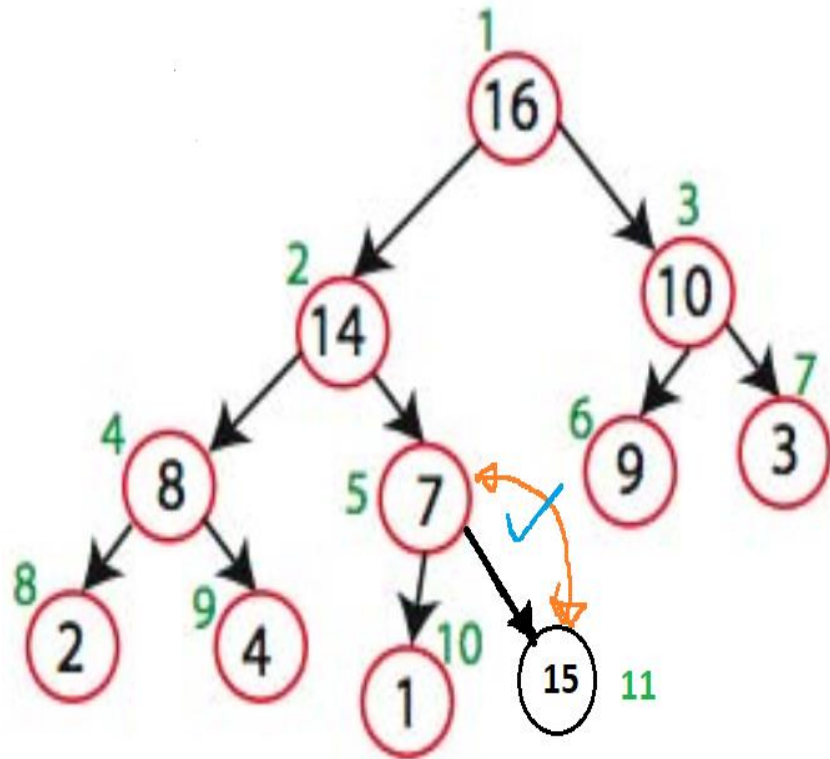
Step-01

InsertHeap(A, 15)

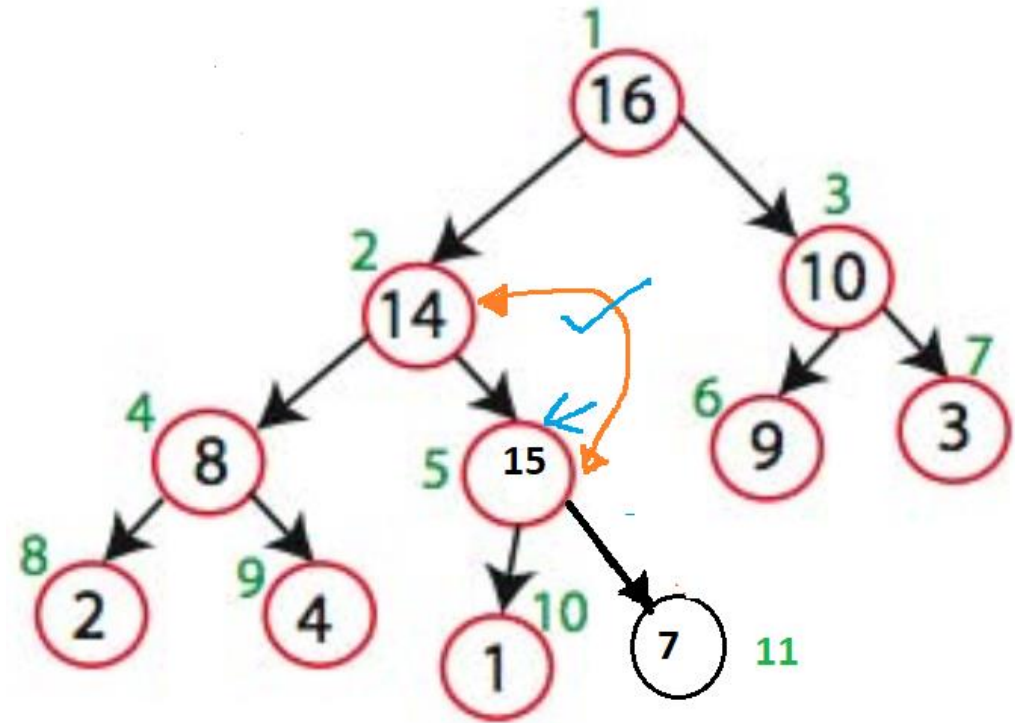


Heap –insertion of a new element

Step-02 InsertHeap(A, 15)

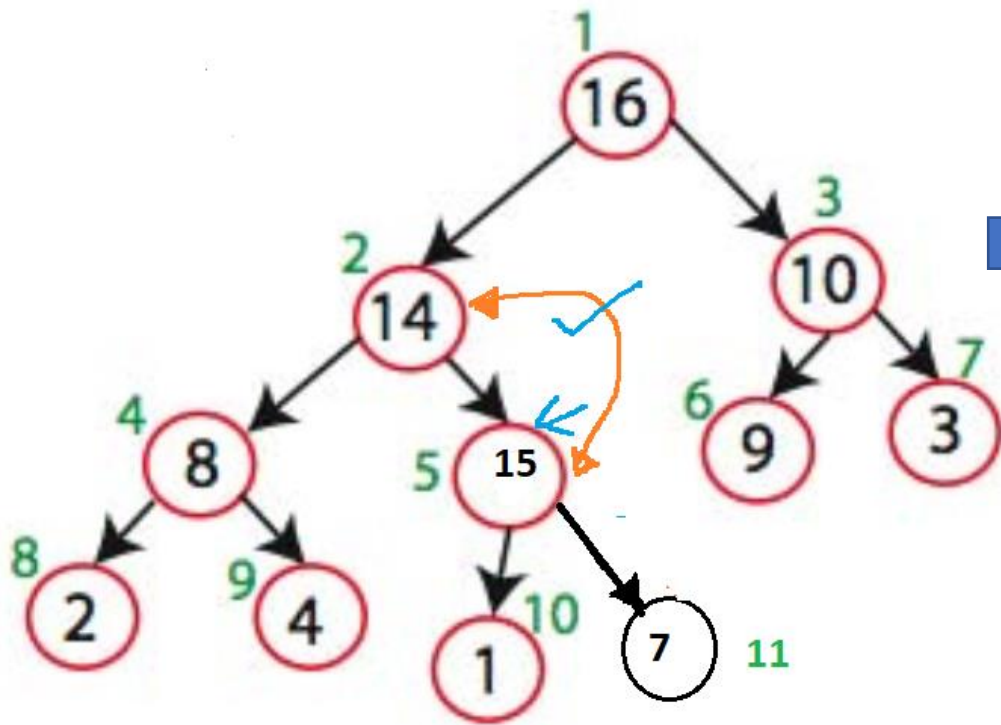


Step-03 InsertHeap(A, 15)

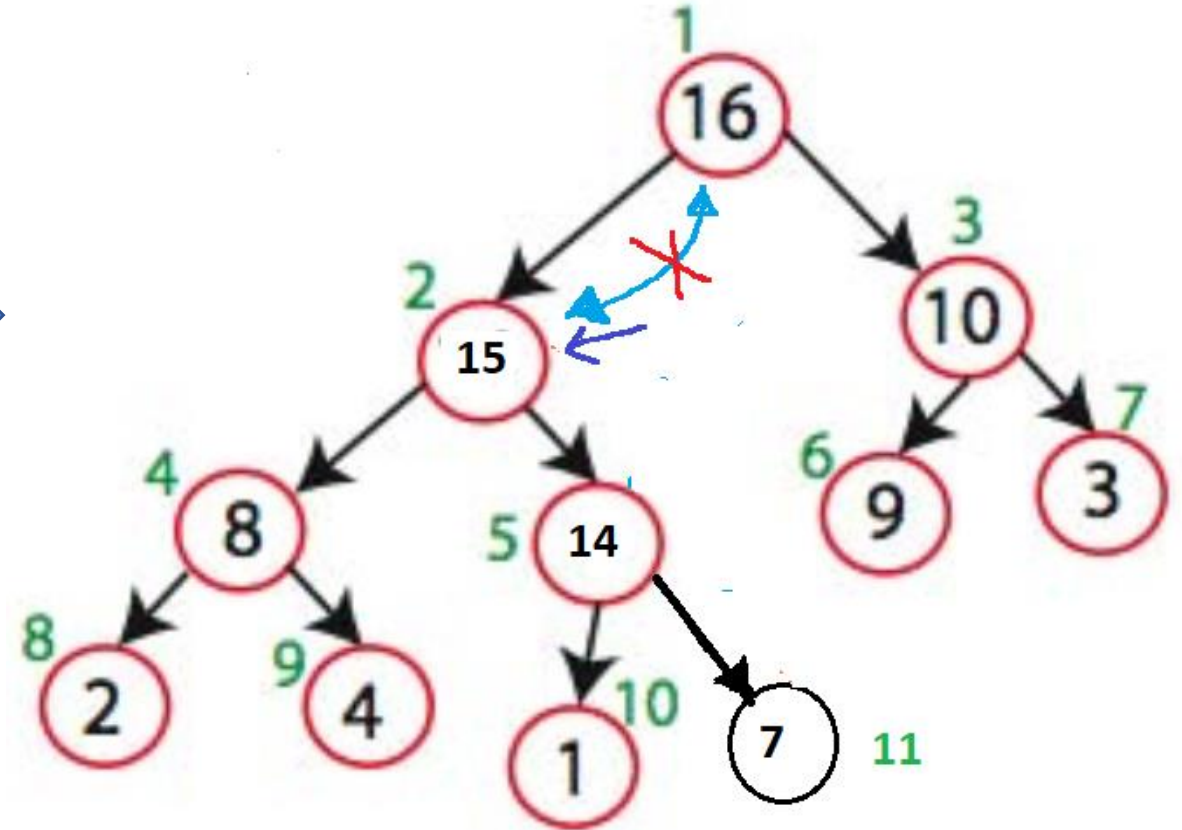


Heap –insertion of a new element

Step-03 InsertHeap(A, 15)

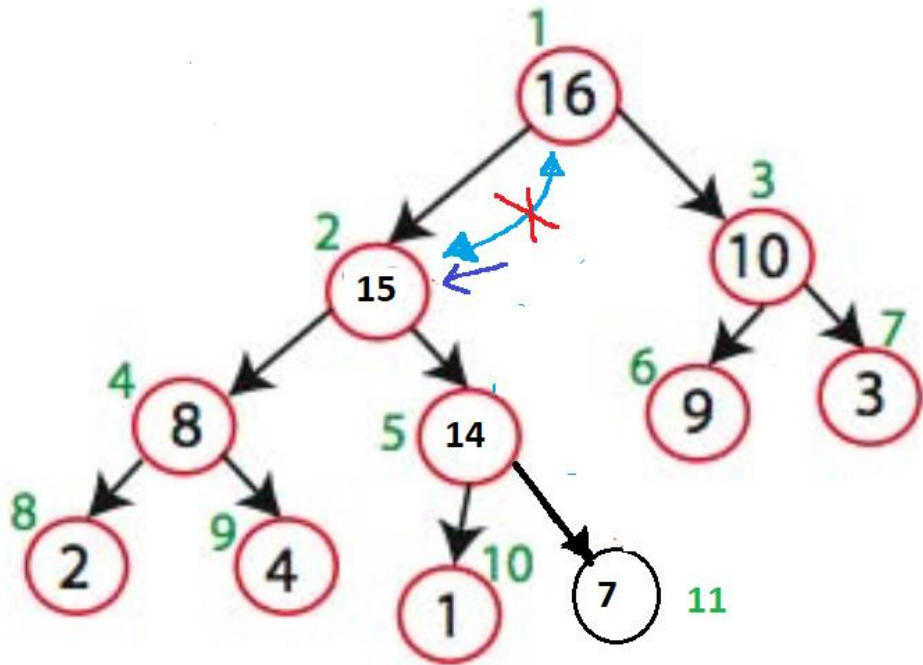


Step-04 InsertHeap(A, 15)

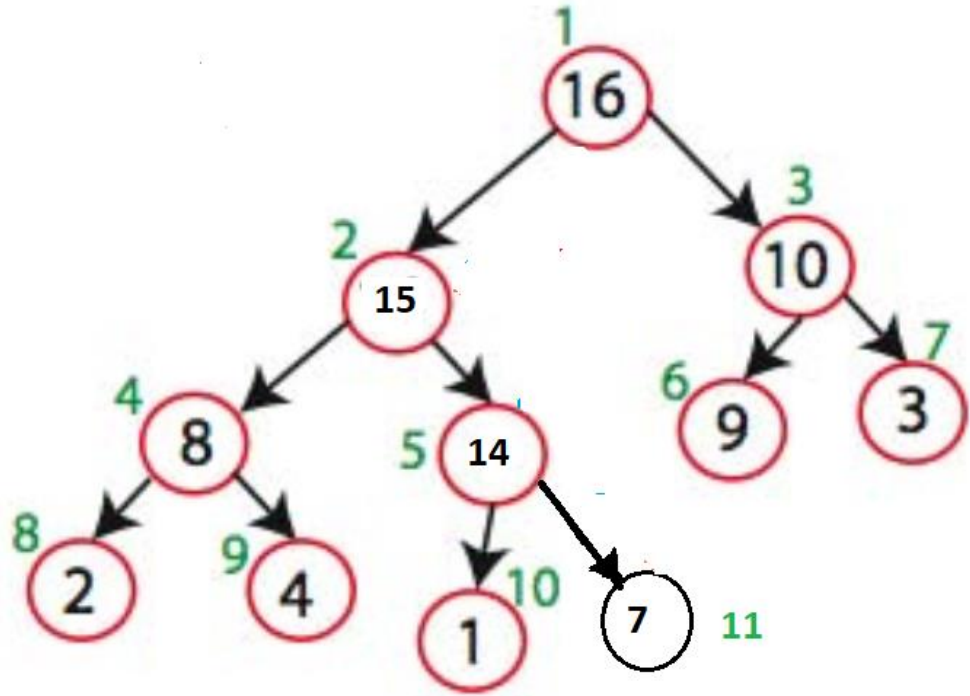


Heap –insertion of a new element

Step-04 InsertHeap(A, 15)



Step-05 InsertHeap(A, 15)
Final



Heap – practice questions

Q1. Consider a max heap, represented by the array: 40, 30, 20, 10, 15, 16, 17, 8, 4. Now consider that a value 35 is inserted into this heap. After insertion, the new heap is? [2 Marks]

Q2. Sort the given elements by creating a min-heap 45, 24, 73, 36, 65, 87, 94, 18, 88. Illustrate with necessary diagrams. [10 marks]

Q3. Arrange the given elements in decreasing order by constructing a heap with the minimum element at the root with top-down and bottom-up approaches. A, B, C, D, E, F, G, H, T, Z, X, W, Q [10 marks]

Q4. Consider the following array of elements. {89, 19, 50, 17, 12, 15, 2, 5, 7, 11, 6, 9, 100}. The minimum number of interchanges needed to convert it into a max-heap is? [3 Marks]